

Relatório Técnico: Modernização DevOps - Loja Veloz

Parte Teórica (Conceituação)

1. Arquitetura de Microsserviços e o Papel do DevOps em Cloud-Native

A arquitetura de microsserviços consiste em desenvolver uma aplicação como um conjunto de pequenos serviços independentes, onde cada um executa seu próprio processo e se comunica por mecanismos leves (como APIs HTTP/REST ou mensageria). No cenário da **Loja Veloz**, os serviços (Pedidos, Pagamentos, Estoque) foram separados para garantir que a falha ou o pico de tráfego de um não derrube o sistema inteiro.

Em um ambiente **cloud-native**, o **DevOps** atua como a ponte que garante que esses múltiplos serviços sejam integrados, testados e entregues (CI/CD) de forma rápida, segura e padronizada, usando práticas de automação, observabilidade e infraestrutura como código (IaC).

2. Containerização: Docker vs Kubernetes

- **Docker:** É a tecnologia que permite empacotar a aplicação e suas dependências em uma imagem imutável e rodá-la em um contêiner (como os **Dockerfiles** que criamos usando Node.js). O **Docker Compose** é excelente para desenvolvimento local, subindo todos os serviços de uma vez.
- **Kubernetes (K8s):** É o orquestrador de contêineres. Enquanto o Docker cria o contêiner, o Kubernetes gerencia milhares deles em produção. Ele sabe quando um contêiner "morreu" e o reinicia (Self-healing), gerencia load-balancing, segredos (Secrets) e escala as instâncias (HPA).
- **Quando utilizar:** Docker (Compose) para a máquina do desenvolvedor (Dev); Kubernetes para ambientes de Homologação e Produção.

3. Fundamentação Teórica

- **Orquestração de Contêineres:** Automatiza o deployment, o gerenciamento, o dimensionamento e a rede dos contêineres. Resolve o problema de "onde meu contêiner vai rodar e como chego até ele?".
- **CI/CD (Integração e Entrega Contínuas):** No modelo distribuído da Loja Veloz, o CI garante que o código não quebre o serviço (testes de unidade, builds de imagem isolados) e o CD gerencia como as novas versões substituem as antigas em produção sem indisponibilidade (usando o Deployment default do K8s com estratégia **RollingUpdate**).
- **Observabilidade:** Diferente de apenas monitorar (verificar se algo caiu), a observabilidade (baseada nos 3 pilares: **Métricas** via Prometheus, **Logs** agregados via ELK/Fluentd, e **Traces Distribuídos** via Jaeger/OpenTelemetry) permite diagnosticar o *porquê* o sistema falhou, mapeando o caminho de um pedido por todos os microsserviços.

4. Fonte de Pesquisa (Case Real)

Case de Referência: O **LuizaLabs** (braço de tecnologia do Magazine Luiza) é um forte exemplo do mercado nacional sobre a migração de monólitos para microsserviços. Eles adotaram Kubernetes e ferramentas cloud-native para suportar eventos de tráfego massivo como a Black Friday. O uso de **Horizontal Pod Autoscaler (HPA)** e **Readiness/Liveness Probes** permitiu ao Magalu escalar seus serviços de forma independente, além de criar pipelines de CI/CD em GitHub/Gitlab que facilitam centenas de deploys diários de maneira segura. *(Fonte inspirada em cases públicos da CNCF e palestras do Magazine Luiza).*

Parte Prática (Justificativa das Decisões)

Para resolver os gargalos da **Loja Veloz**, foi construído um MVP funcional:

◆ Ambiente Local com Docker Compose

- **Decisão:** Criação de um `docker-compose.yml` central.
- **Por quê:** O time agora pode iniciar o ecossistema inteiro (PostgreSQL, RabbitMQ, Pedidos, Pagamentos) rodando apenas `docker-compose up -d`. Foram configuradas redes (`bridge`) exclusivas e volumes (`pgdata`) para garantir a persistência dos dados e a comunicação sem expor todas as portas diretamente para a máquina do host.

◆ Containerização e Segurança

- **Decisão:** Uso de `Dockerfiles Multi-stage`.
- **Por quê:** No primeiro estágio (`builder`), instalamos todas as dependências do Node.js. No segundo estágio, geramos uma imagem enxuta de produção. Além disso, criamos um **usuário não-root** (`nodejs`) no contêiner para evitar escalonamento de privilégios, mitigando falhas graves de segurança.

◆ Kubernetes para Produção

- **Manifests (`Deployment` e `Service`):** Organizados para isolar configuração de acesso do gerenciamento de réplicas.
- **ConfigMaps e Secrets:** As credenciais e variáveis sensíveis foram abstraídas, seguindo o padrão 12-Factor App ("Store config in the environment").
- **Probes:** Adicionados `Liveness` e `Readiness` no endpoint `/health`. O K8s só enviará tráfego para o Pod de "Pedidos" quando ele estiver 100% pronto (Readiness), e irá reiniciá-lo automaticamente se parar de responder (Liveness).

◆ CI/CD com GitHub Actions

- **Decisão:** Pipeline em `.github/workflows`.
- **Por quê:** A cada Push/PR na `main`, o pipeline automatiza a instalação de dependências, faz o build da imagem Docker utilizando `Caching` de camadas (para acelerar o CI) e submete para um Docker Registry, validando o código antes do deploy.

◆ Estratégia de Deploy e Escalabilidade

- **Deploy Estratégico:** Adotado o **Rolling Update** (padrão do K8s Deployments). Isso permite que novos Pods substituam gradualmente os antigos, garantindo que o sistema não saia do ar durante atualizações (Zero Downtime).
- **Escalabilidade (HPA):** O `HorizontalPodAutoscaler` configurado (`pedidos-hpa`) foi setado para escalar de 2 até 10 réplicas quando o uso de CPU ultrapassar 70%. Isso é crucial para as campanhas promocionais e picos de tráfego previstos no desafio.